UNIT II:
Packages, Interfaces and Fundamental Classes

Dr. Mrs. V. V. Hanchate

UNIT II: PACKAGES, INTERFACES AND FUNDAMENTAL CLASSES

String, Introduction to Packages, Types of Fundamental Classes, String Handling: Creating Format String, Exception Handling, Nested Classes, Threads, Collections and Maps, Collections, Networking: Socket Programming, URL class

Course Objective

- **Introduce** the concept of package, exception, thread, fundamental classes and networking using Java

Course Outcome

- **Explain** the concept of package, exception, thread, fundamental classes and networking using Java (Bloom's Level 2 : Understand)

STRINGS

- String is a sequence of characters, for e.g. “Hello” is a string of 5 characters.
- Strings are special in Java.
- In java, string is an immutable object which means it is constant and can cannot be changed once it has been created.
- Strings in Java are not primitive types (like int, char, etc). Instead, all strings are objects of a predefined class named String.
- And, all string variables are instances of the String class. To create the string objects you need not to use ‘new‘ keyword. Where as to create other type of objects you have to use ‘new’ keyword

Creating a String:

There are two ways to create a String in Java

1. String literal
2. Using new keyword

STRING LITERAL:

String s1 = "abc";
String s2 = "xyz";

Using "new" :

```
String s5 = new String("abc");
```

Using new with array:

String arr[];

String arr[] = new String(3);

Using String buffer class :

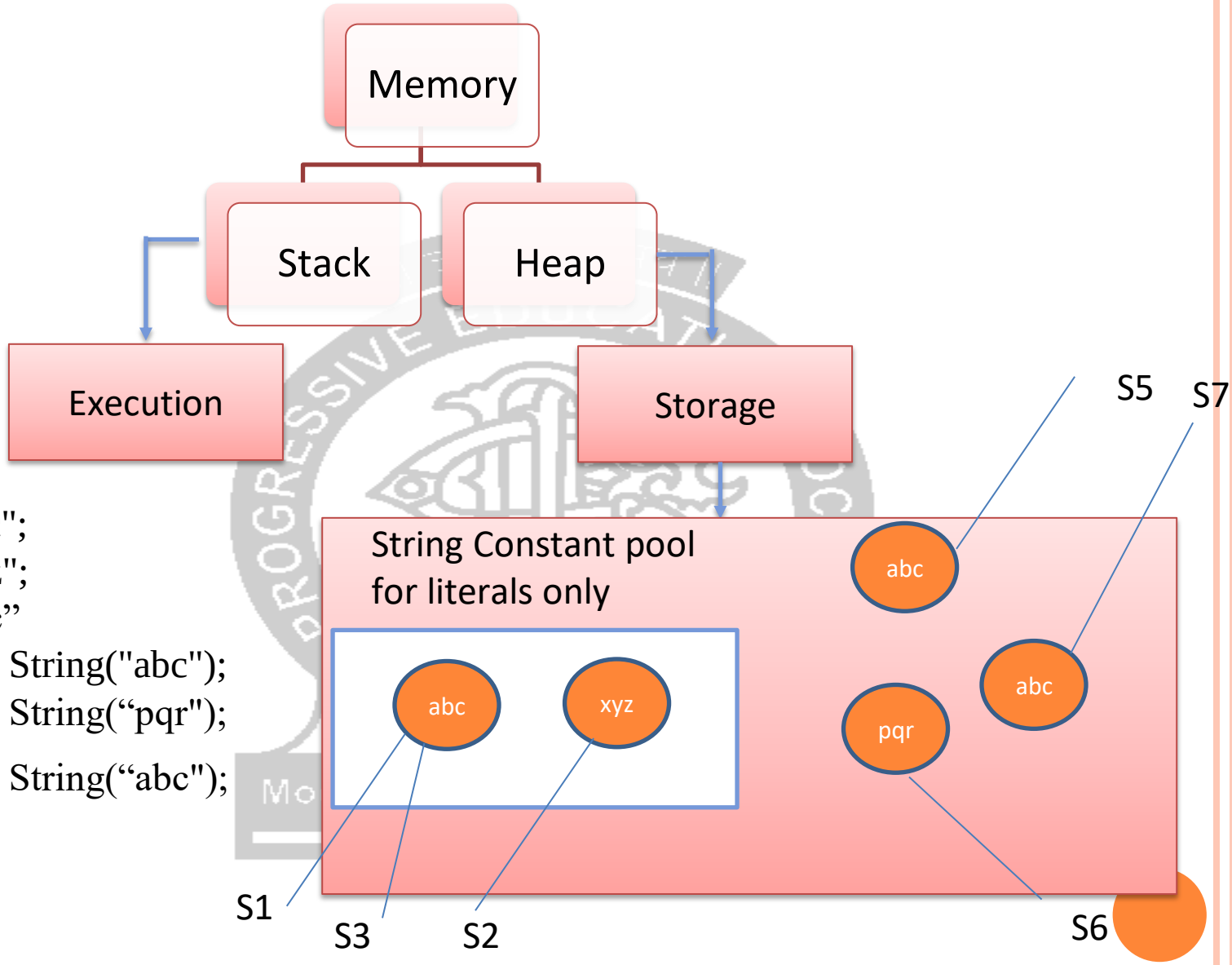
```
String S7 = new String(new StringBuffer());
```

String buffer creates a string of flexible length that can be modified in terms of both length and contents.

Few such methods are:

1. S7.stChartAt(n,'x'); : modifies nth character to x
2. S7.append(S2) : Appends string S2 to S7 at the end.
3. S7.insert(n,S2) : inserts string S2 at the position n of the string S7

Memory allocation in String




```
String s1 = "abc";
String s2 = "xyz";
String s3 = “abc”
String s5 = new String("abc");
String s6 = new String(“pqr");
String s7 = new String(“abc");
```

- Fact about String Constant Pool is that, pool space is allocated to an object depending upon it's content. There will be no two objects in the pool having the same content.
- This is what happens when you create string objects using string literal,
- When you create a string object using string literal, JVM first checks the content of to be created object. If there exist an object in the pool with the same content, then it returns the reference of that object. It doesn't create new object. If the content is different from the existing objects then only it creates new object.
- But, when you create string objects using new keyword, a new object is created whether the content is same or not.

JAVA STRING OPERATIONS

Length of a String :

```
class Main {
 public static void main(String[] args) {

   // create a string
   String get = "Hello! World";
   System.out.print("String: " + get);

   // get the length of greet
   int length = get.length();
   System.out.print("Length: " + length);
 }
}
```

Join two Strings:

```
class Main {
  public static void main(String[] args) {

    // create first string
    String first = "Java ";
    System.out.print("First String: " + first);

    // create second
    String second = "Programming";
    System.out.print("Second String: " + second);

    // join two strings
    String joinedString = first.concat(second);
    System.out.print("Joined String: " +
joinedString);
  }
}
```

We can also join two strings using the + operator in Java

Compare two Strings:

```
class Main {
  public static void main(String[] args) {

    // create 3 strings
    String first = "java programming";
    String second = "java programming";
    String third = "python programming";

    // compare first and second strings
    boolean result1 = first.equals(second);
    System.out.print("Strings first and second are equal: " + result1);

    // compare first and third strings
    boolean result2 = first.equals(third);
    System.out.print("Strings first and third are equal: " + result2);
  }
}
```

Methods of Java String

Methods	Description
substring()	returns the substring of the string
replace()	replaces the specified old character with the specified new character
charAt()	returns the character present in the specified location
getBytes()	converts the string to an array of bytes
indexOf()	returns the position of the specified character in the string
compareTo()	compares two strings in the dictionary order
trim()	removes any leading and trailing whitespaces
format()	returns a formatted string
split()	breaks the string into an array of strings
toLowerCase()	converts the string to lowercase
toUpperCase()	converts the string to uppercase
valueOf()	returns the string representation of the specified argument
toCharArray()	converts the string to a char array

WRITE A PROGRAM TO SORT THE STRINGS

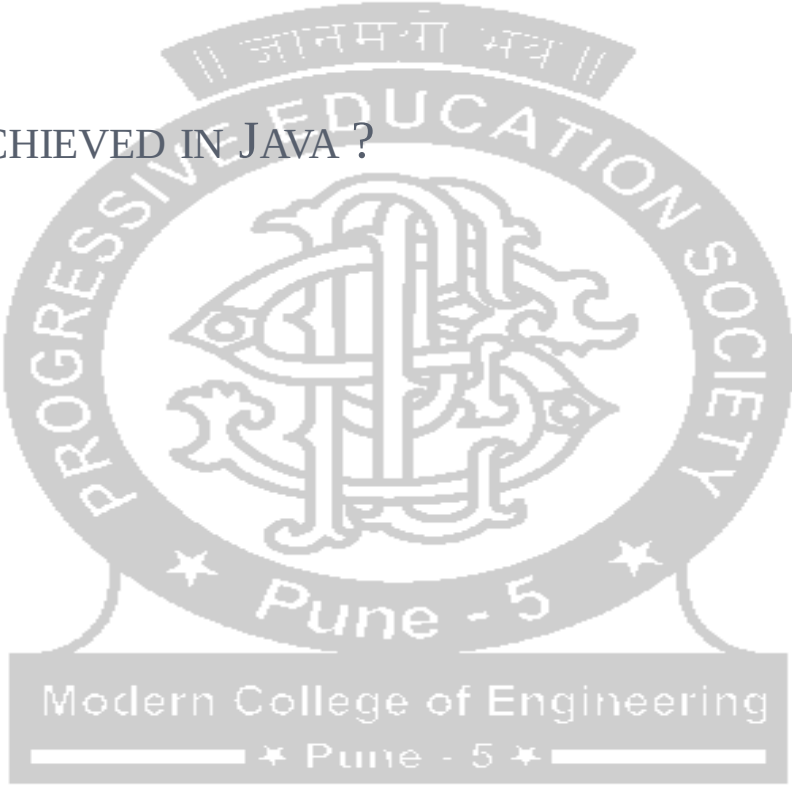
```
import java.util.Scanner;
public class String_sort {
public static void main(String[]
args) {
Scanner s = new Scanner(System.in);
System.out.print("Enter no of
strings:");
int size = s.nextInt();
String a[]= new String[size];
System.out.print("Enter
strings:");
for(int i=0;i<size;i++ )
{
a[i] = s.next();
}
//String temp = 0;
for(int i=0;i<size-1;i++ )
{
for(int j=i+1;j<a.length;j++ )
{
if(a[i].compareTo(a[j]) > 0)
{
String temp = a[i];
a[i] = a[j];
a[j] = temp;
}
}
}
System.out.print("\n"+"Sorted
strings:");
for(int i=0;i<size;i++ )
{
System.out.print(a[i]+ "\n");
}
}
}
```

PACKAGES

- Need of Package

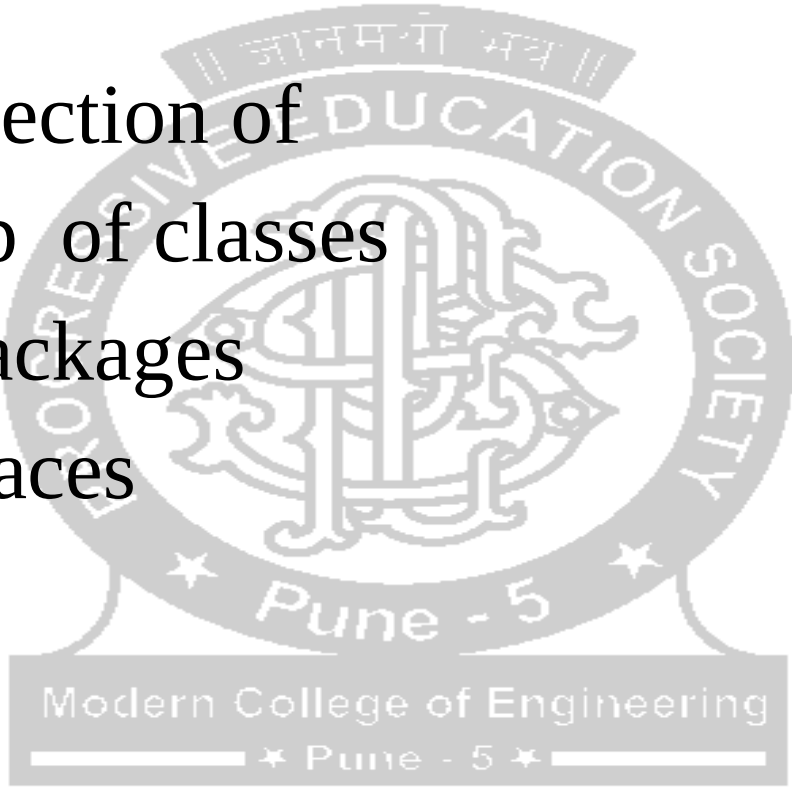
HOW REUSABILITY IS ACHIEVED IN JAVA ?

- ✓ Using extends
- ✓ Using implements



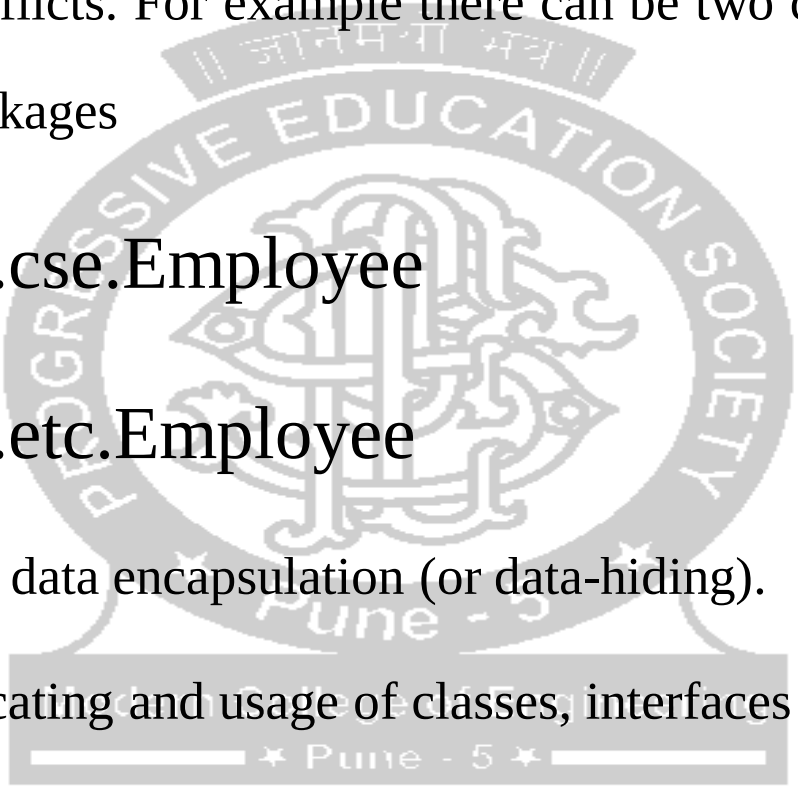
Package is collection of

- Group of classes
- Subpackages
- Interfaces



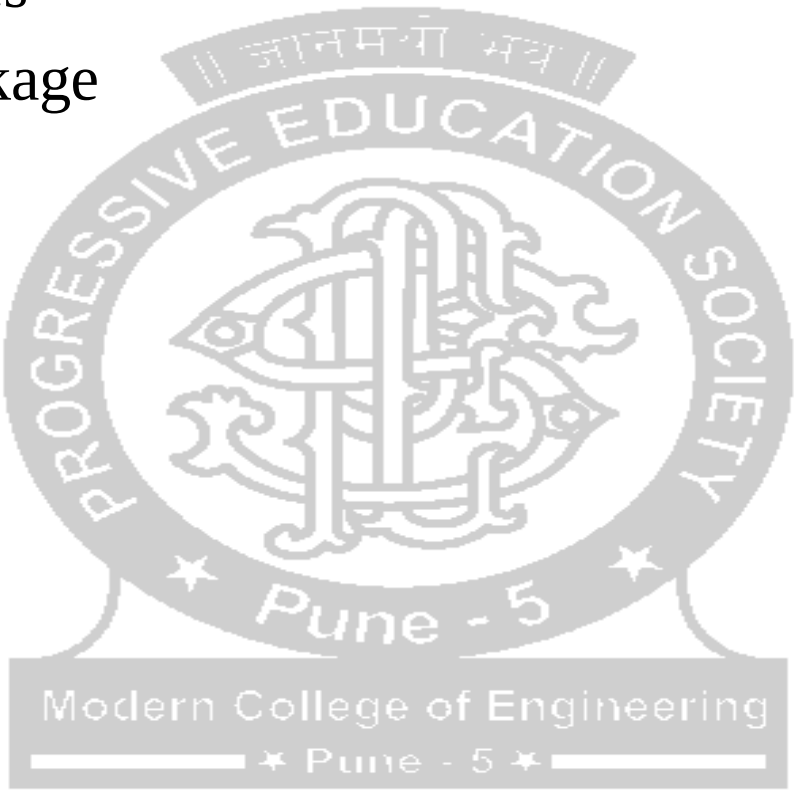
Use of Packages

- The classes can be easily reused
- To prevent name conflicts. For example there can be two classes with name Employee in two packages
  - college.staff.cse.Employee
  - college.staff.etc.Employee
- Can be considered as data encapsulation (or data-hiding).
- Making searching/locating and usage of classes, interfaces easier

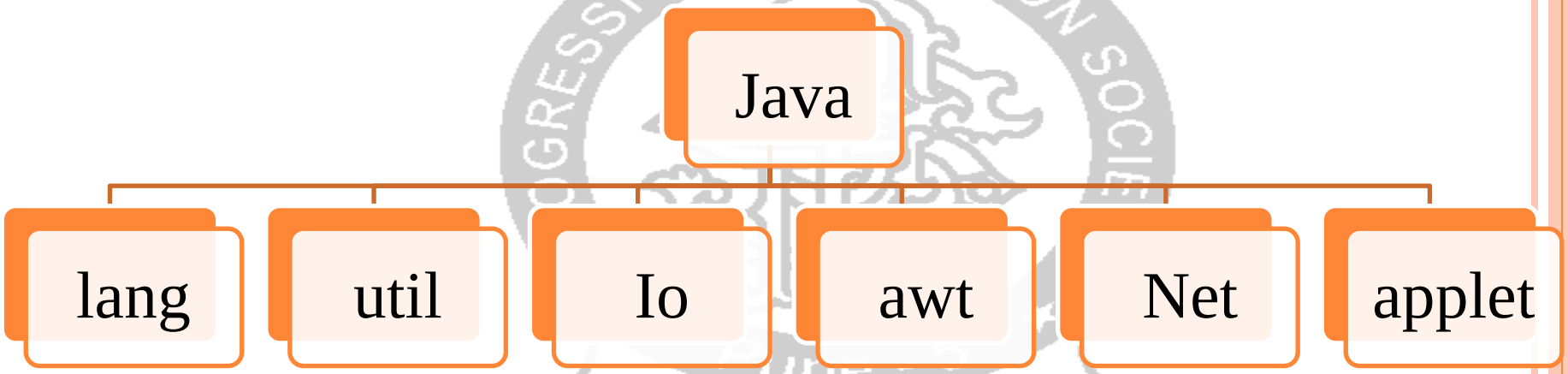


Types of Packages

- Java API Packages
- User defined package



Java API Package



Making Use Of Java API Package

import java.util.Scanner;

Making Use of Java API Package util

```
public class Factorial {

        public static void main(String[] args) {
        Scanner in = new Scanner(System.in);

        System.out.println("Enter the No to calculate its factorial");
        double no;

        no = in.nextDouble();

        double fact = 1;

        for(int i = 1; i<=no; ++i)
        {
                fact *=i;
        }
        System.out.println("Factorial =" + fact);
    }
```

Creating a User Defined Package

```
package <package name>;
public class <class name> {
   ----------------
}
public class <class name> {
   ----------------
}
```

EXAMPLE

```
package myPackage;


public class MyClass {
   public void getNames(String s) {
         System.out.println(s);
   }
}
```

```
import myPackage.MyClass;
public class PrintName {
   public static void main(String args[]) {
        String name = “Modern College";
        MyClass obj = new MyClass();
        obj.getNames(name);
    }
}
```

Scope of the class declared in Package

- Public
- Private

Advantages of Packages

- Programmers can define their own packages to bundle a group of classes/interfaces, etc.
- Since the package creates a new namespace there won't be any name conflicts with names in other packages.
- Using packages, it is easier to provide access control
- It is also easier to locate the related classes

Hiding a Class Declared in Package

- If class declared inside the package is not public(means private), then it is hidden from outside the package.
- It can be used by all the classes in the same package.
- E.g.

```
package p1;
public class X{
        Body of X
}

class Y{
        Body of Y
}
```

Class X visible outside the package as access specifier is public

Class Y hidden outside the package as access specifier is not public

Exception Handling in Java

Compile time error and runtime error.

- Any program has two types of statements

1. int a=1; / k=i+j; ---- normal type

2. k=i/j; ---- critical type

The second statement is critical because it can take any value. If j = 0 then the statement is critical.

There are two types of errors.

1. Compile time errors

2. Runtime errors.

COMPILE TIME ERROR AND RUNTIME ERROR

- Any program has two types of statements

  1. int a=1; / k=i+j; ---- normal type
  2. k=i/j; ---- critical type

The second statement is critical because it can take any value. If j = 0 then the statement is critical.

There are two types of errors.

1. Compile time errors
2. Runtime errors.

Need of Exception handling

```
Statement 1;
Statement 2;
Statement 3;
Statement 4;
Statement 5;
 k= i/j;
 Statement 7;
Statement 8;
Statement 9;
```

Statements 1 to 5 will get executed. If j=2; then next all statements will be executed. But if j=0;
Then exception has occurred and it will not execute the remaining statements. Thus we need to handle these exceptions.

What is Exception handling?

This is one of the powerful mechanism to handle the runtime errors so that normal flow of applications can be maintained.

Types of Exceptions

- **Checked Exceptions:**

Compiler knows that there is an error and indicates you as you are writing the code.

- **Unchecked Exceptions:**

Compiler doesn't know there is an error. After execution of program you come to know there is an error.

Runtime Exceptions in java

1	ArithmeticException Arithmetic error, such as divide-by-zero.
2	ArrayIndexOutOfBoundsException Array index is out-of-bounds.
3	ArrayStoreException Assignment to an array element of an incompatible type.
4	ClassCastException Invalid cast.
5	IllegalArgumentException Illegal argument used to invoke a method.
6	IllegalMonitorStateException Illegal monitor operation, such as waiting on an unlocked thread.
7	IllegalStateException Environment or application is in incorrect state.
8	IllegalThreadStateException Requested operation not compatible with the current thread state.
9	IndexOutOfBoundsException Some type of index is out-of-bounds.
10	NegativeArraySizeException Array created with a negative size.
11	NullPointerException Invalid use of a null reference.
12	NumberFormatException Invalid conversion of a string to a numeric format.
13	SecurityException Attempt to violate security.
14	StringIndexOutOfBounds Attempt to index outside the bounds of a string.

Checked Exceptions in Java

Sr.No.	Exception & Description
1	ClassNotFoundException Class not found.
2	CloneNotSupportedException Attempt to clone an object that does not implement the Cloneable interface.
3	IllegalAccessException Access to a class is denied.
4	InstantiationException Attempt to create an object of an abstract class or interface.
5	InterruptedException One thread has been interrupted by another thread.
6	NoSuchFieldException A requested field does not exist.
7	NoSuchMethodException A requested method does not exist.

Handling Exceptions

- These exceptions can be handled by Try block and catch block.

```
try
    {
        Statements that can cause error/exception;
    }
catch(Exception e)
  {
        statements those handle the exception if occurred
   }
```

TRY BLOCK AND CATCH BLOCK

```
import java.util.Scanner;
public class Demo_exception {

public static void main(String[] args) {
int k=0;
Scanner s = new Scanner(System.in);
System.out.print("Enter number for i");
int i = s.nextInt();
System.out.print("Enter number for j");
int j = s.nextInt();
try
{
k =i/j;
}
catch(ArithmeticException e)
{
System.out.print("\n"+"Number cannot be divided by 0");
}
System.out.print("\n"+"The result is: "+k);
}

}
```

TRY WITH MULTIPLE CATCH

```
import java.util.Scanner;
public class Demo_exception {

public static void main(String[]
args)
{
int k=0;
Scanner s = new Scanner(System.in);
System.out.print("Enter number for
i");
int i = s.nextInt();
System.out.print("Enter number for
j");
int j = s.nextInt();
try
{
k =i/j;
int a[]= new int[4];
for(int p=0;p<=4;p++)
{
a[p]=p++;
}
for(int p=0;p<=4;p++)
{
System.out.print("\n"+a[p]);
}}
catch(ArithmeticException e)
{
System.out.print("\n"+"Number
cannot be divided by 0");
}
catch(ArrayIndexOutOfBoundsExceptio
n e)
{
System.out.print("Size of array is
more");
}
catch(Exception e)
{}
System.out.print("\n"+"The result
is: "+k);
}}
```

FINALLY BLOCK

```
IMPORT JAVA.UTIL.SCANNER;
PUBLIC CLASS DEMO_EXCEPTION {

PUBLIC STATIC VOID MAIN(STRING[] ARGS) {
INT K=0;
SCANNER S = NEW SCANNER(SYSTEM.IN);
SYSTEM.OUT.PRINT("ENTER NUMBER FOR I");
INT I = S.NEXTINT();
SYSTEM.OUT.PRINT("ENTER NUMBER FOR J");
INT J = S.NEXTINT();
TRY
{
K =I/J;
INT A[]= NEW INT[4];
FOR(INT P=0;P<=4;P++)
{
A[P]=P++;
}
FOR(INT P=0;P<=4;P++)
{
SYSTEM.OUT.PRINT("\N"+A[P]);
}
}
```

```
catch(ArithmeticException e)
{
System.out.print("\n"+"Number cannot be divided by 0");
}
catch(ArrayIndexOutOfBoundsException e)
{
System.out.print("Size of array is more");
}
catch(Exception e)
{}

finally
{
s.close();
System.out.print("\n"+"Thank you: See you another day");
}
System.out.print("\n"+"The result is: "+k);
}
}
```

THROW

```
import java.util.Scanner;
public class Exception_demo {

        public static void main(String[] args)
        {
                Scanner s = new Scanner(System.in);
                int i,j,k=0;
                i=8;
                try
                {
                        System.out.print("\nEnter number");
                        j= s.nextInt();
                   k=i/j;
                   System.out.print(k);
                   throw new ArithmeticException();
                  }
                finally
                {
                        s.close();
                        System.out.print("\n Thank you");
                }}}
```

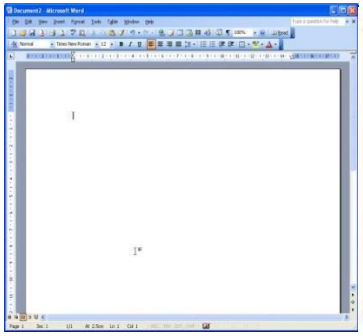
THREAD

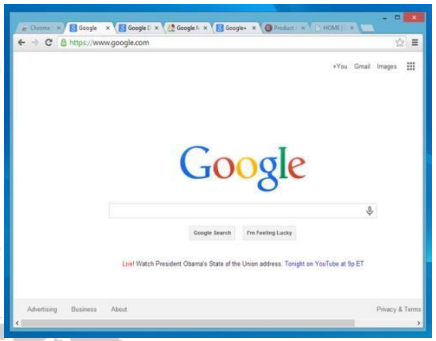
॥ जानमग्ने मय ॥
PROGRESSIVE EDUCATION SOCIETY
Pune - 5
Modern College of Engineering
Pune - 5

MULTITASKING

- Multitasking is when a single CPU performs **several tasks(processes)** at the same time.
- To perform multitasking, the CPU switches among these tasks very **frequently** so that user can interact with each program simultaneously.
- In a multitasking operating system, several users can **share the system** simultaneously.
- CPU allocates different memory space for all the processes that are running.

Example of Multitasking





Process1

Process2

Process3

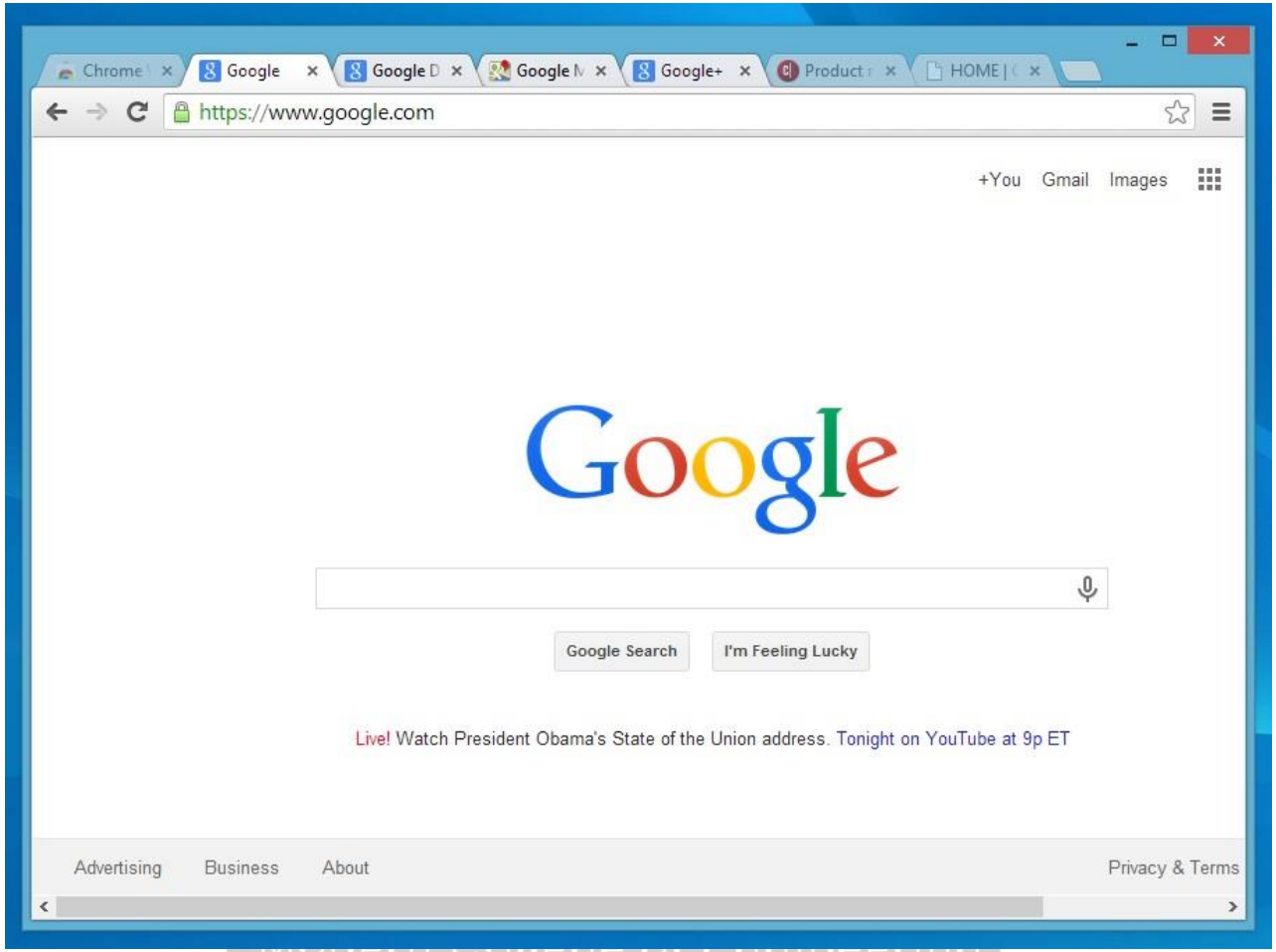
Process4

Process5

Operating System

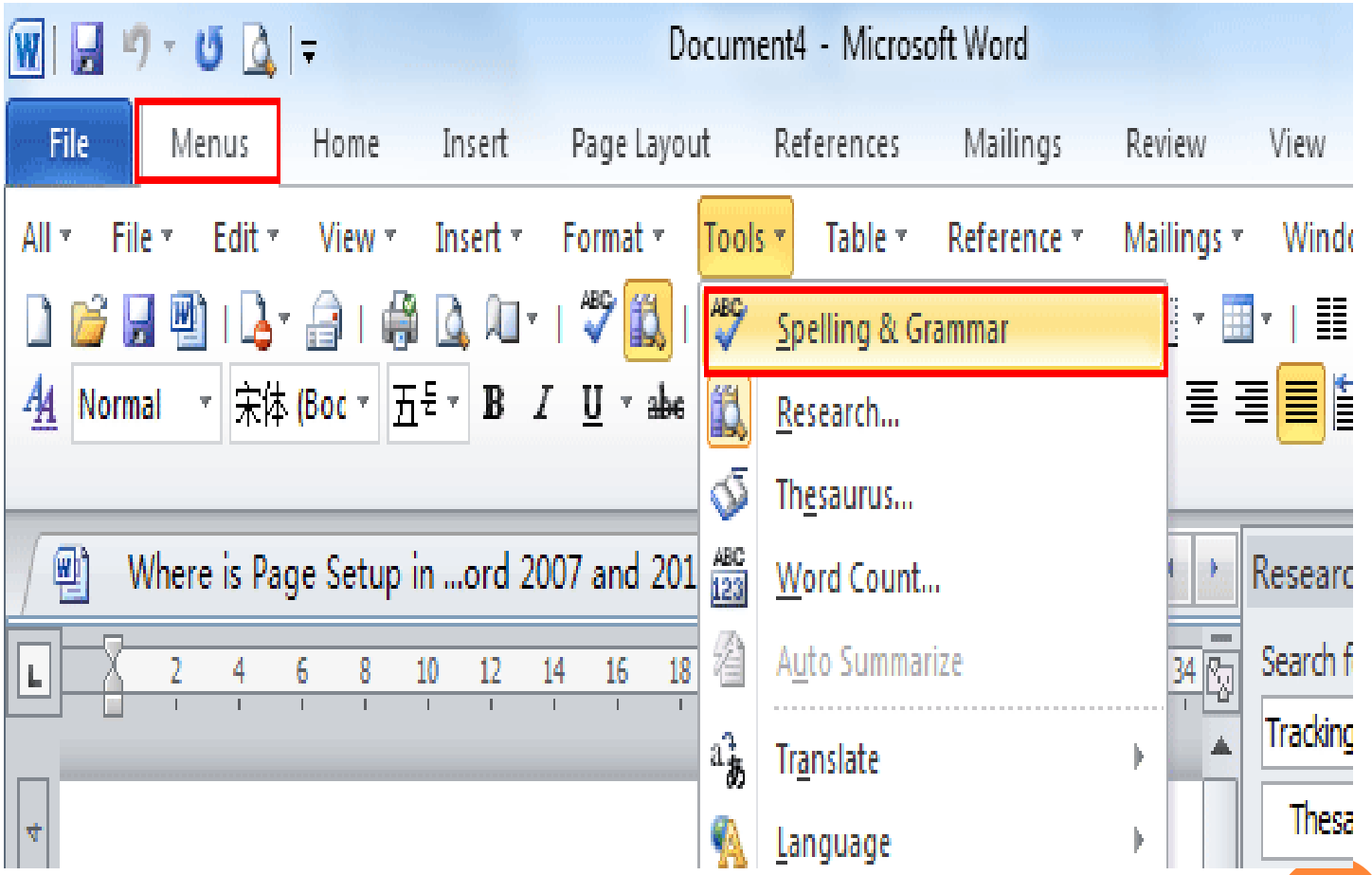
CPU

Multithreading Example 1



Chrome is a process which internally runs different processes. (tabs). These different processes(tabs) of one single process(chrome) are called threads of that main process(chrome). This is an example of multithreading.

Multithreading Example 2



Example of single threading.

Difference between Multitasking and Multithreading

Multitasking	Multithtreding
Multitasking let CPU to execute multiple tasks at the same time.	Multithreading let CPU to execute multiple threads of a process simultaneously.
In multitasking CPU switches between programs frequently.	In multithreading CPU switches between the threads frequently.
In multitasking system has to allocate separate memory and resources to each program that CPU is executing.	In multithreading system has to allocate memory to a process, multiple threads of that process shares the same memory and resources allocated to the process.

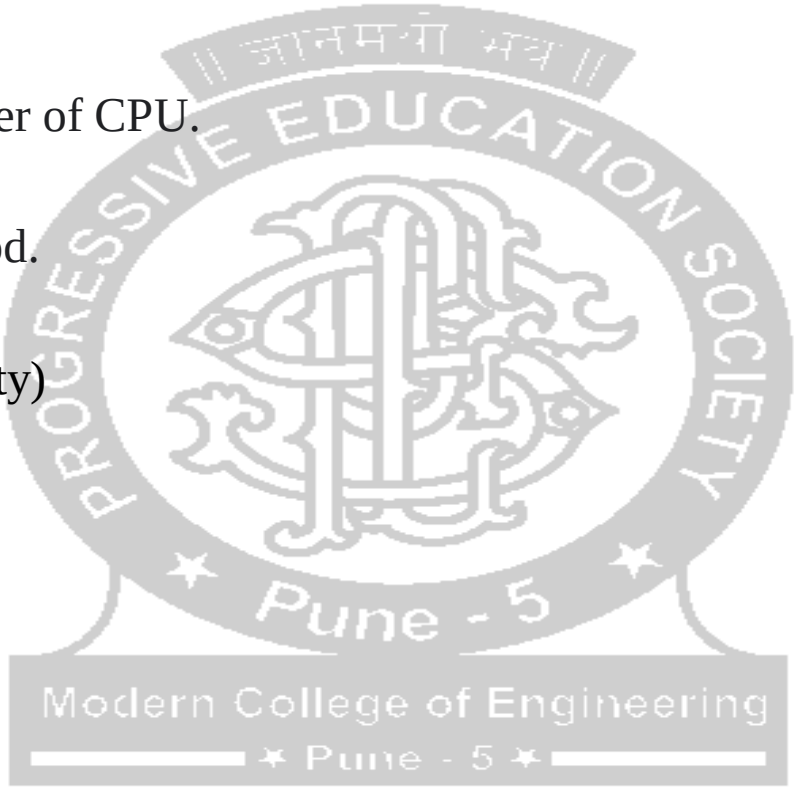
WHAT IS A THREAD?

- Threads are lightweight sub-processes, they share the common memory space.
- Threads in java are subprograms of a main application program and share same memory space, they are known as lightweight threads or light weight processes.
- Threads can be created in two ways:
    1. By **implementing** the **Runnable interface**.
    2. By **extending** the Thread class
- A thread goes through various stages in its life cycle

Before studying threads lets understand why we need threads.

Why Threads

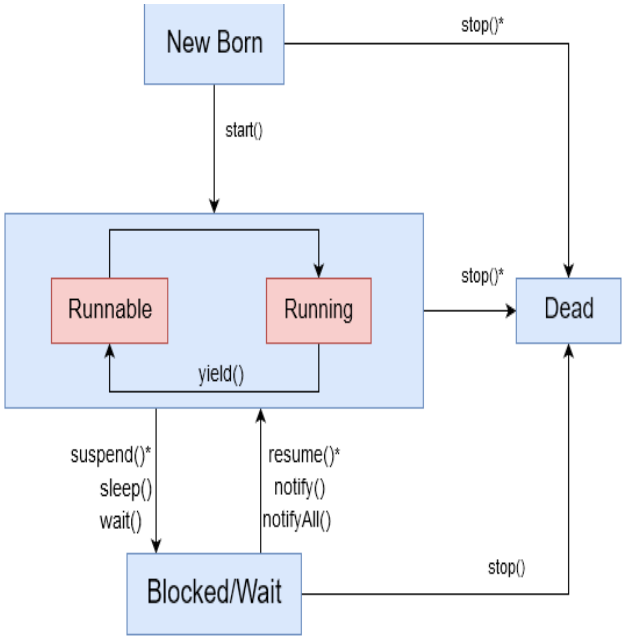
1. Threads allows a program to operate more efficiently by doing multiple things at the same time.
2. Using complete power of CPU.
3. Async task in Andriod.
4. Gaming (Call of Duty)
5. Web Applications



By extending the Thread class

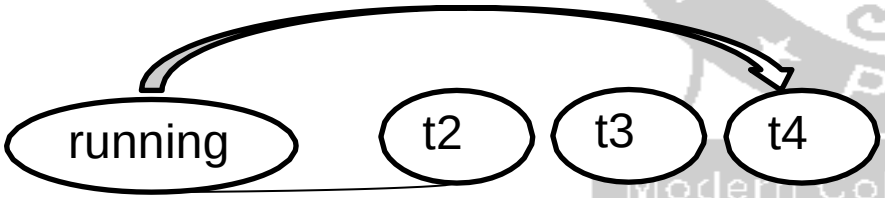
Method	Description
getName()	Obtain thread's name
getPriority()	Obtain thread's priority
isAlive()	Determine if a thread is still running
Join()	Wait for a thread to terminate
Run()	Entry point for the thread
Sleep()	Suspend a thread for a period of time
Start()	Start a thread by calling its run method

LIFE CYCLE OF A THREAD



- During lifetime of a thread it enters many several states as shown in diagram.
- **New born:**
- When we create a thread object, the thread is born and is in new born state. Two things can be done now:
- Schedule it to run using method start() or kill it using method stop().
- If scheduled to run then it moves into runnable state.

In this state if thread wants other thread to go into the running state then that is done by calling method **yield()**



Runnable state:

In this state the thread is ready for execution and waiting for processor. That is the thread has joined the queue of threads. And they will enter in running state onfirst come first basis.

LIFE CYCLE CONTINUED….

Running state:

In this state the processor has given time to the thread for its execution. The running thread can be suspended and resumed again by using **suspend() or resume()** methods. The thread can be made to sleep too using **sleep (time)** method. Here time is in **milliseconds**. When in sleep it means that the thread is out of queue during this period. The thread enters runnable state the moment time is elapsed. The thread can be put into wait/bocked state using methods wait() and scheduled to run again using notify().

Blocked state:

Thread is in blocked state when it is prevented to enter runnable and later running state.

Dead state:

After the execution of the thread it goes into dead state. We can put thread in dead state by using method stop().

By extending the Thread class

Declaring class:

```
class A extends Thread
{
        }
```

Implementing run method

This run method is to be defined in class itself.

```
Public void run()
{
}
```

Starting a new Thread:

This is to be written in main method

```
A obj = new A();
obj.start();
```

Above two lines can be written in a single line as

```
new A().start();
```

Let us understand practical implementation of thread.

Thread Exceptions..

- Sleep(milliseconds) method cannot be used in program as it is.
- This is because a sleeping thread cannot deal with resume() method as a sleeping thread cannot receive any instructions. The same is true with suspend method.
- In order to make a thread receive our instruction when sleeping we use Thread Exceptions.
- The thread exceptions are try block and catch block.
- Syntax:

  try { sleep(500) ; } / try { Thread.sleep(500); }

- catch block can take various forms. We will discuss three of them.

  catch (Exception e) { } Any other

  catch (ThreadDeath e) { } killed thread

  catch (InterruptedException e) { } cannot handle in the current state.

```
package thread_Demo;
class Hi extends Thread
{
public void run()
{
for(int i=0;i<5;i++)
{
System.out.println("Hi");
try
{
Thread.sleep(500);
}
catch(Exception e)
{}
}
}
}
```

```
class Hello extends Thread
{
public void run()
{
for(int i=0;i<5;i++)
{
System.out.println("Hello");
try
{
Thread.sleep(500);
}
catch(Exception e)
{}
}
}
}
public class Thread_demo {
public static void main(String[]
args) {
        Hi obj1 = new Hi();
        Hello obj2 = new Hello();
        obj1.start();
        obj2.start();
}}
```

THREAD PRIORITY

- The threads we saw so far are of same priority.
- We can assign priority to threads. The processor will schedule the threads depending upon their priority,
- Java permits us to set priority of a thread using **setPriority()** method.
- Every thread has a priority which is represented by the integer number between 1 to 10.
- Thread class provides 3 constant priorities:

  public static int MIN_PRIORITY: The value of MIN_PRIORITY is 1

  public static int NORM_PRIORITY: It is the normal priority of a thread. The value of it is 5.

  public static int MAX_PRIORITY: The value of MIN_PRIORITY is 10

- We can also set the priority of thread between 1 to 10. This priority is known as custom priority or user defined priority.

THREAD PRIORITY…

Syntax:

object. setPriority(**int** a)

a: It is the priority to set this thread to. The method does not return a value.

Exception:

IllegalArgumentException: This exception throws if the priority is not in the range MIN_PRIORITY to MAX_PRIORITY.

The priority is to be set in main method only. Ex: consider object

of a thread is t1. t1.setPriority(Thread.MIN_PRIORITY);

t1.setPriority(Thread.MAX_PRIORITY);

t1.setPriority(Thread.NORM_PRIORITY);

Userdefined priority: t1.setPriority(4); the value can bet between 1 to 10 only

Let us see the practical implementation.

```
package thread_Demo;
class Hi extends Thread
{
public void run()
{
for(int i=0;i<5;i++)
{
System.out.println("Hi");
try
{
Thread.sleep(500);
}
catch(Exception e)
{}
}
}
}
```

```
class Hello extends Thread
{
public void run()
{
for(int i=0;i<5;i++)
{
System.out.println("Hello");
try
{
Thread.sleep(500);
}
catch(Exception e)
{}
}}}
public class Thread_demo {
public static void main(String[] args) {
// TODO Auto-generated method stub
        Hi obj1 = new Hi();
        Hello obj2 = new Hello();
  obj2.setPriority(Thread.MAX_PRIORITY);
  obj1.setPriority(3);
        obj1.start();
        obj2.start();
}}
```

Implementing Runnable Interface

- Runnable is an interface.
- The only function declared in this interface is run().
- The runnable interface is implemented by the class to create a thread.
- The thread goes into runnable state only if its started using start() method.
- But start method is of class Thread and not interface Runnable.
- Then what is the solution?

Let us understand through an example.

```
class A implements Runnable
{
        public void run()
        {
        for(int i =0; i<5;i++)
        {
                System.out.print("\nGood");
                try
                {
                Thread.sleep(500);
                }
                catch(Exception e) { }
         }
        }
}
```

```
class B implements Runnable
{       public void run()
        {
        for(int i =0; i<5;i++)
        {
        System.out.print("\nMorning");
                try
                {
                Thread.sleep(500);
                }
                catch(Exception e) { }
        } } }
public class Demo_thread {

        public static void main(String[] args)
        {
                Runnable obj = new A();
                Runnable obj1 = new B();
                Thread t1 = new Thread(obj);
                Thread t2 = new Thread(obj1);
                t1.start();
                t2.start();
        } }
```

USE OF VOLATILE IN MULTITHREADING

```
class VolatileThread extends Thread {
    private volatile boolean flag = true;

    public void run() {
        while (flag) {
            System.out.println("Running");
        }
    }

    public void stopRunning() {
        flag = false;
    }
}

public class MultithreadingApp {

public static void main(String[] args)throws InterruptedException  {
// TODO Auto-generated method stub
VolatileThread example = new VolatileThread();
        example.start();

        // Sleep for 1 second
        Thread.sleep(1000);

        example.stopRunning();
      }

}
```

Nested Classes

- Also called as inner classes

```
class Java_Outer_class{
 //code
 class Java_Inner_class{
  //code
 }
}
```

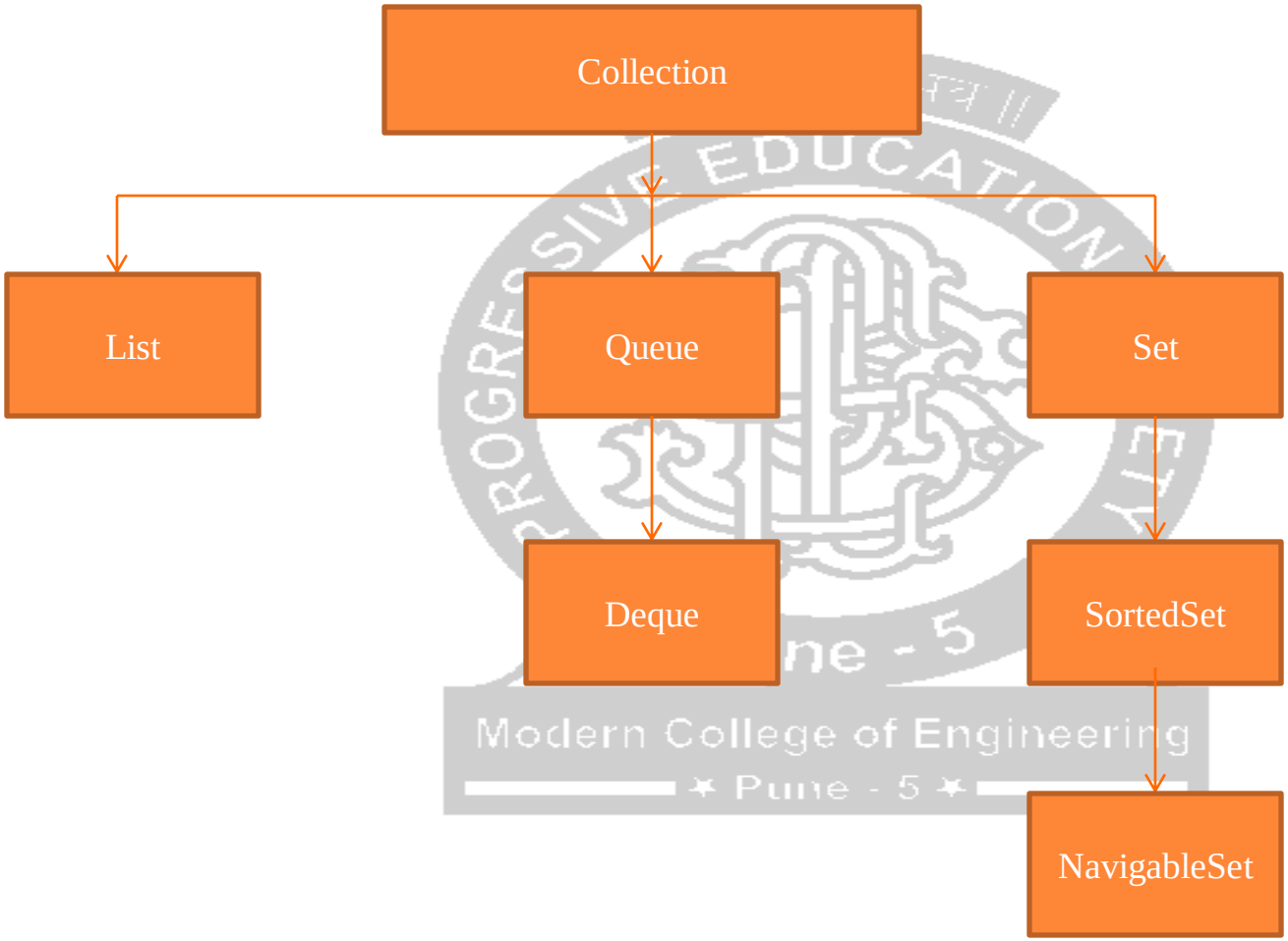
Advantages of Java Nested Classes

- For representing a particular type of relationship
- To improve readability of code
- Optimization of Code

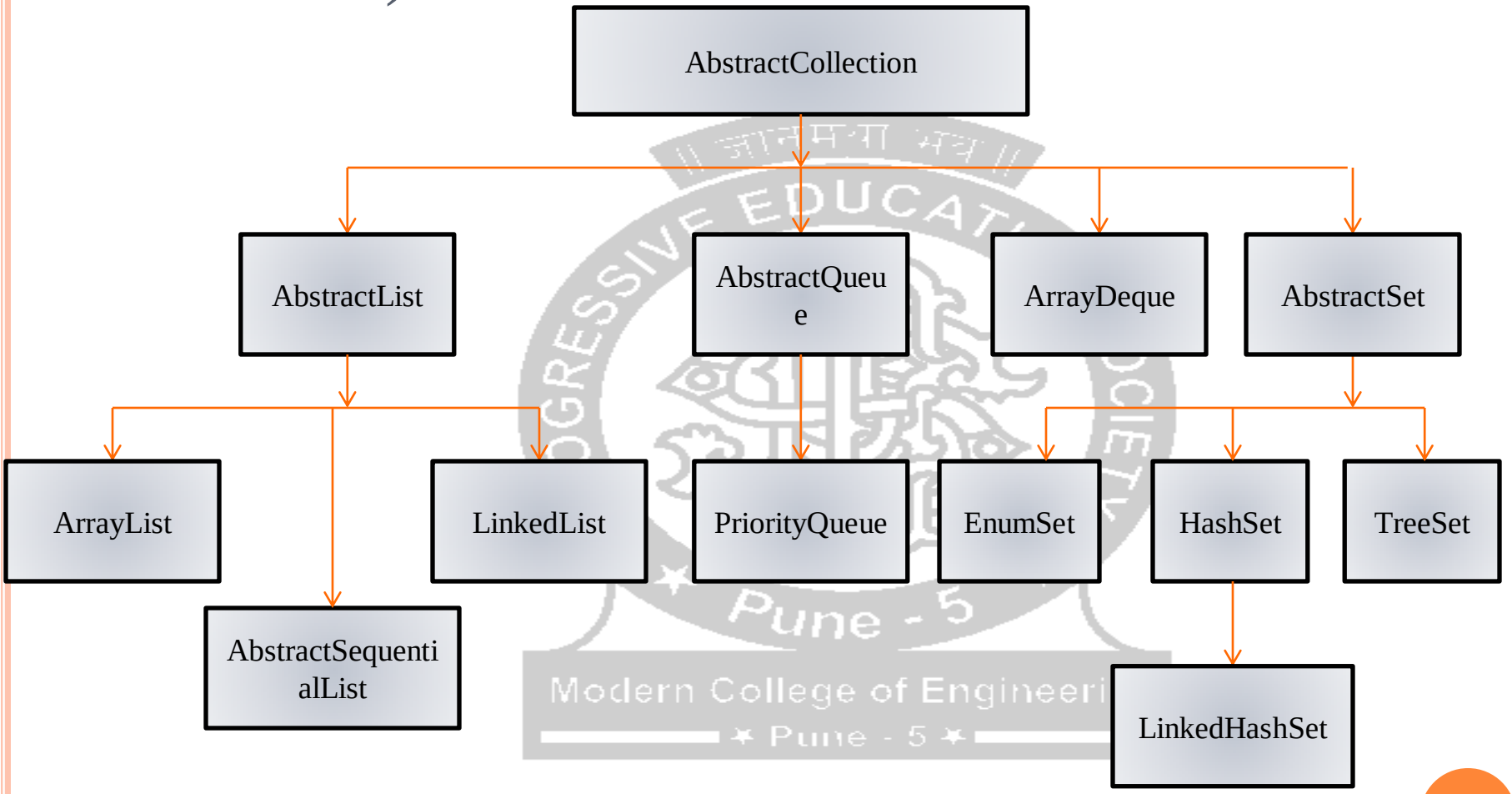
Collection and Maps

- Collection and Map are interface defined into util package
- Root interface of Collection hierarchy
- Interfaces supported by Collection
  - Set
  - List
  - Queue
  - Deque
- Classes supported by Collection
  - ArrayList
  - Vector
  - LinkedList
  - PriorityQueue
  - HashSet
  - LinkedHashSet
  - TreeSet

Hierarchy Of Collection(Interfaces Available)



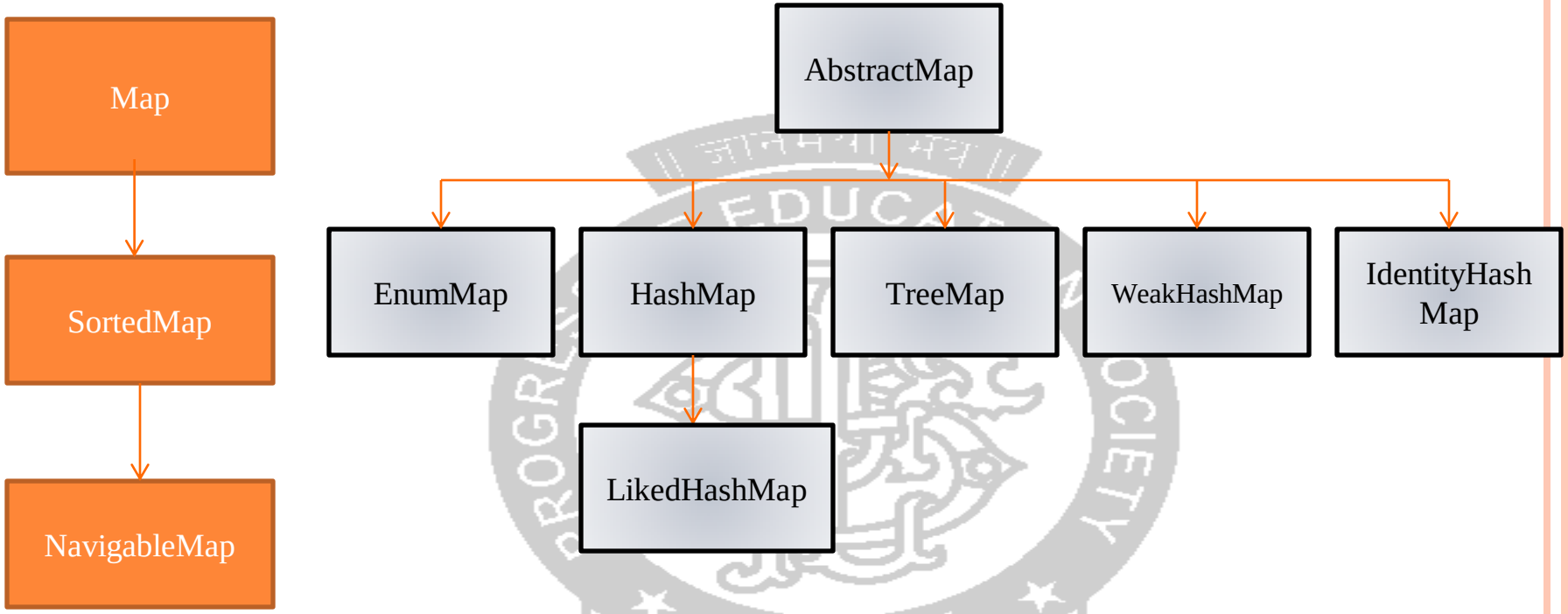
Hierarchy Of Collection(Classes Available)



Map Interface

- Generic object that stores association between keys and value
- Both key and values are object
- Unique key but multiple values possible

Map Hierarchy



Networking

- java.net
- Protocol Supported
  - Transmission Control Protocol**(TCP)**
  - User Datagram Protocol**(UDP)**
- Classes supported by TCP
  - URL
  - URLConnection
  - Socket
  - ServerSocket
- Classes supported by UDP
  - DatagramPacket
  - DatagramSocket
  - MulticastSocket

URL (UNIFORM RESOURCE LOCATOR)

- Points to a resource on the World Wide Web

Example :

```
import java.net.*;
public class URLClassDemo{
public static void main(String[] args){
try{
URL url=new URL("https://moderncoe.edu.in:100/research-center");

System.out.println("Protocol: "+url.getProtocol());
System.out.println("Host Name: "+url.getHost());
System.out.println("Port Number: "+url.getPort());
System.out.println("File Name: "+url.getFile());

}catch(Exception e){System.out.println(e);}
}
}
```


Output :

```
Protocol: https
Host Name: moderncoe.edu.in
Port Number: 100
File Name: /research-center
```

InetAddress Class

- Represents an IP address
- Useful to get an IP address of any host

InetAddress Class Methods

Method	Does this
boolean equals(Object obj)	Compares this object with the reference
byte [] getAddress()	Return raw IP address of this object
static InetAddress getByAddress(byte [] addr)	Return InetAddress for given address
static InetAddress getByAddress(String host, byte [] addr)	Creates an InetAddress based on provided hostname and IP address
static InetAddress getByName(String host)	Determine the IP address of the host, given the host name
String getHostAddress()	It returns the IP address in string
String getHostName()	It returns Host name
static InetAddress getLocalHost()	Returns local host
int hashCode()	Returns a hashcode for this IP address
String toString()	Converts this IP address to String

INETADDRESS FACTORY METHODS

- The InetAddress class has no visible constructors. The InetAddress class has the inability to create objects directly, hence factory methods are used for the purpose.
- They are static methods in class to return an object

Method	Description
public static InetAddress getByName(String host) throws UnknownHostException	Returns the instance of InetAddress containing IP and Host name of host represented by host argument.
public static InetAddress getLocalHost() throws UnknownHostException	Returns the instance of InetAddress containing the local hostname and address.
public static InetAddress[] getAllByName(String hostName) throws UnknownHostException	Returns the array of the instance of InetAddress class which contains IP addresses.

```
import java.net.*;

public class InetAddressDemo{
public static void main(String[] args) {
try {
        InetAddress address = InetAddress.getLocalHost();
        System.out.println("Address :" + address);

        address = InetAddress.getByName("www.moderncoe.edu.in");
        System.out.println("Address :" + address);

        InetAddress SW[] = InetAddress.getAllByName("www.moderncoe.edu.in");
        for( int i= 0; i < SW.length; ++i)
        {
                System.out.println(SW[i]);
        }
}
catch(UnknownHostException e) {}
}
}
```

Output :

Address :DESKTOP-Q22KFL9/172.16.4.207
Address :www.moderncoe.edu.in/174.138.120.32
www.moderncoe.edu.in/174.138.120.32

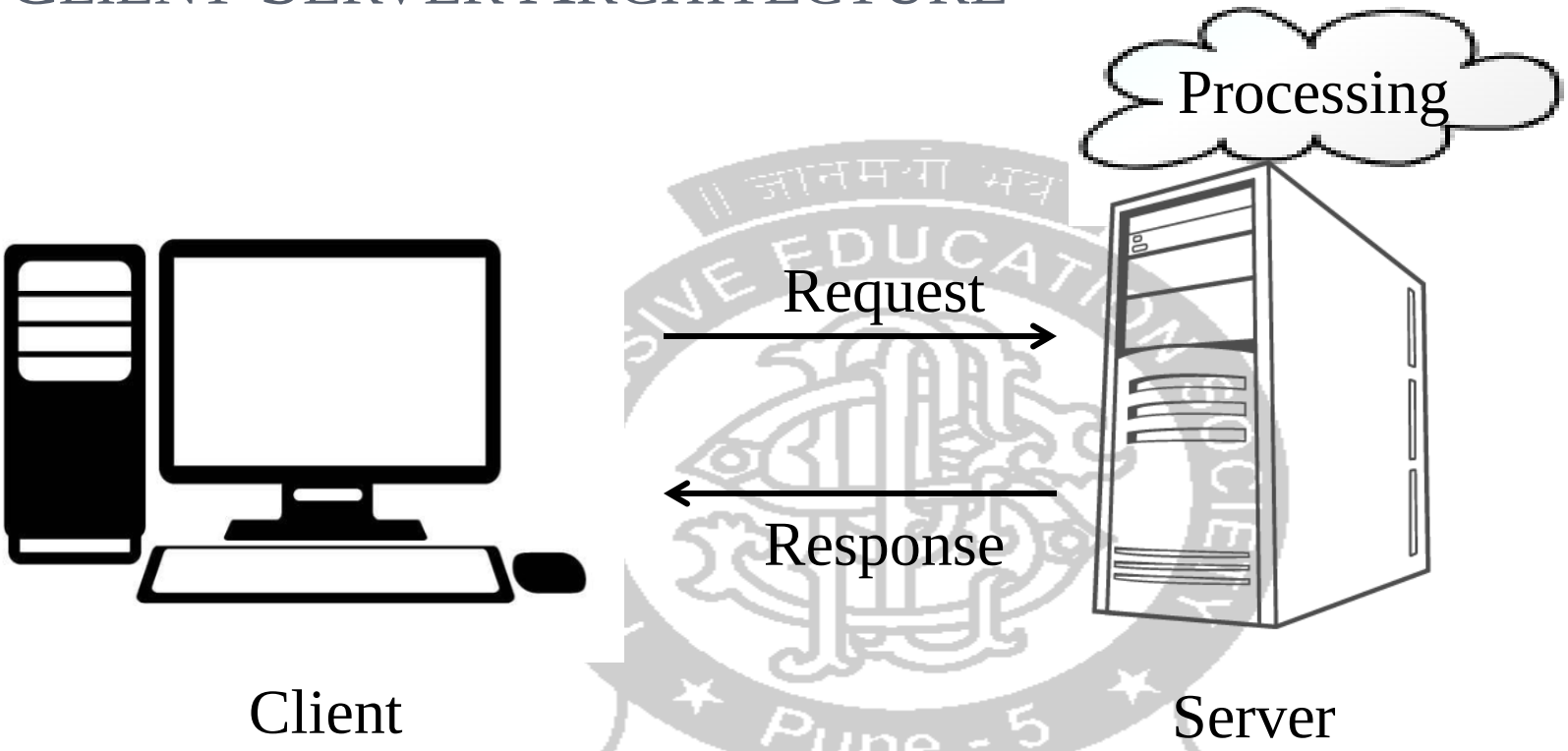
Instance Method of InetAddress Class

Method	Description
boolean equals(Object other)	Rturns true if other object has same IP address
byte [] getAddress()	Return the byte array that represent the objectt's IP address in network byte order
String getHostAddress()	Returns a string that represents host address associated with InetAddress object
String getHostName()	Return a string that represent host name
boolean isMulticastAddress()	Return true if address is multicast
String toString()	Return a string that lists host name and IP address for the convenience.

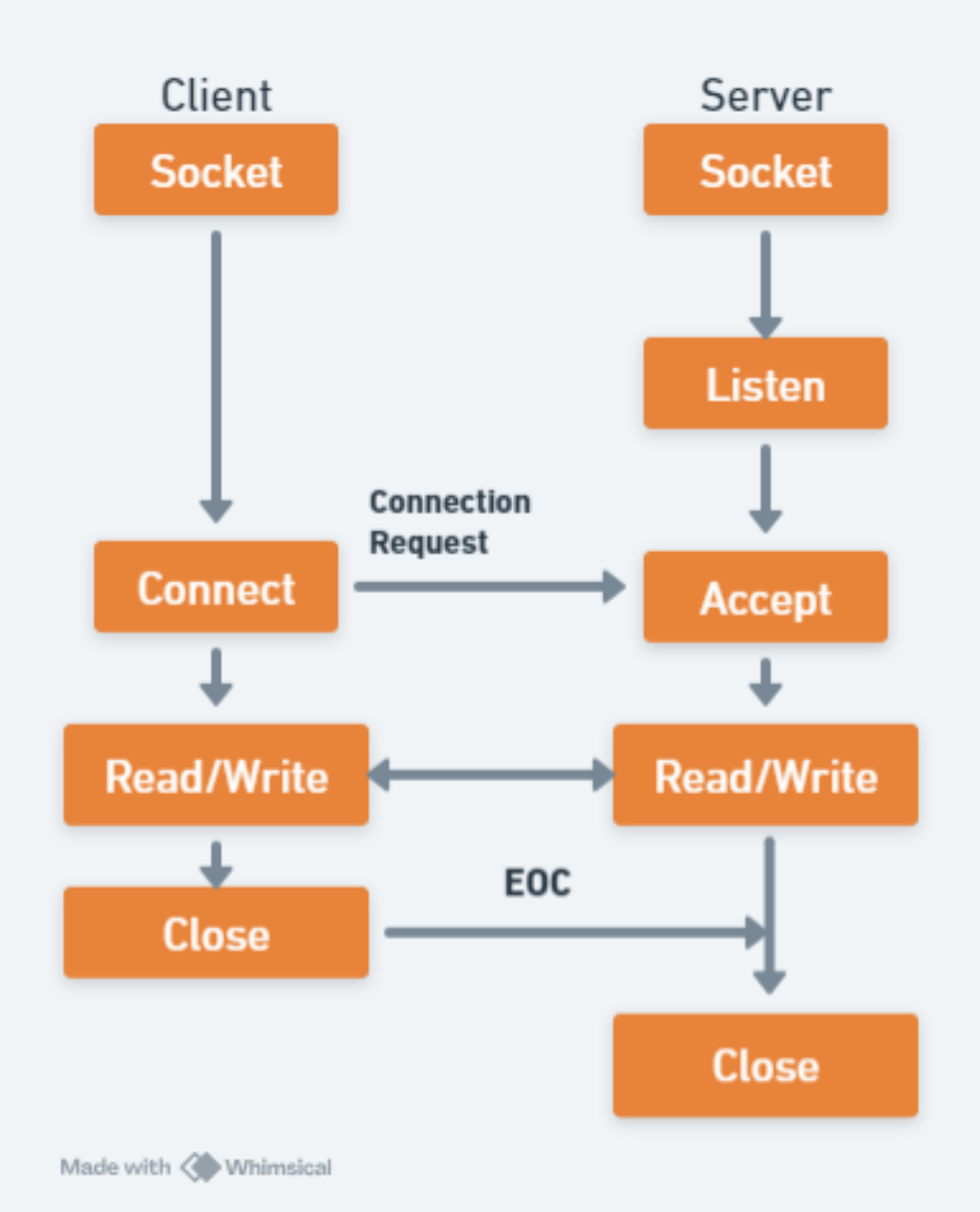
SOCKET

- An internet protocol(IP) port on specific host machine is called Socket
- Two Types
  - Connection oriented Sockets (based on TCP)
    - Provides the guarantee of data delivery in correct order
  - Connectionless Socket (based on UDP)
    - No guarantee of data delivery and the correct order

Client Server Architecture



TCP/IP Socket



Socket Class Methods

Method	Description
public InputStream getInputStream()	returns the InputStream attached with this socket.
public OutputStream getOutputStream()	returns the OutputStream attached with this socket.
public synchronized void close()	closes this socket

ServerSocket Methods

Method	Description
public Socket accept()	returns the socket and establish a connection between server and client.
public synchronized void close()	closes the server socket.

```
import java.io.*;
import java.net.*;

public class MyServer {
public static void main(String[] args){
try{
ServerSocket ss=new ServerSocket(1000);
Socket s=ss.accept();//establishes connection

DataInputStream MyDataInputStr=new
DataInputStream(s.getInputStream());

String   str=(String)MyDataInputStr.readUTF();
System.out.println( "Msg Received : " + str);

ss.close();

}catch(Exception e){System.out.println(e);}
}
}
```

```
import java.io.*;
import java.net.*;

public class MyClient {
public static void main(String[] args) {
try{
Socket s=new Socket("localhost",1000);

DataOutputStream dout=new DataOutputStream(s.getOutputStream());

dout.writeUTF("Welcome to Socket Programming");

dout.close();
s.close();

}catch(Exception e){System.out.println(e);}
}
}
```

Thank you..